

에이전트란 무엇인가

챗봇과 무엇이 다른가, 무엇으로 이루어지고, 어떻게 만드는가

입문자를 위한 에이전트 한 바퀴. **개념**부터 **구성요소**, **설계**와 **도구**
·MCP·스킬, 그리고 실제 무인 시스템 사례까지 한 흐름으로 본다.

대상 에이전트를 처음 접하는 입문자

출처 Anthropic, Building Effective AI Agents (2024) 외 공식 문서

제작 AGENT 입문 · 모두의연구소

이 자료의 큰 흐름

개념에서 시작해 **직접 만드는 감각**까지 이어지도록 구성했다.

01

에이전트란 무엇인가

챗봇·단일 호출과 무엇이 다른지, 왜 **자율 실행 주체**라 부르는지 짚는다.

02

무슨 문제에 맞나

에이전트가 **잘 맞는 일**과 **오히려 과한 경우**를 가르는 판단 기준을 살피고, 도입 여부를 정하는 법을 본다.

03

무엇으로 이루어지나

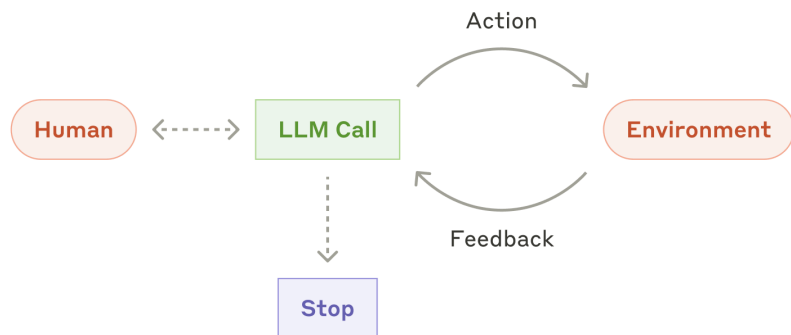
모델·도구·메모리·루프·환경 다섯 조각이 어떻게 맞물려 도는지 살핀다.

04

어떻게 잘 만드나

설계 5단계와 **결정론·LLM 분리**, 도구·MCP·스킬, 실제 무인 시스템 사례까지.

챗봇은 답하고, 에이전트는 끝낸다



자율 에이전트 루프(Human-LLM-Environment 순환). 출처: Anthropic, Building Effective AI Agents, 2024

01 무엇을 보나

사용자 지시로 시작해 **행동(Action)**과 **피드백(Feedback)**이 **환경**과 오가며 도는 **자율 루프**다. 챗봇의 한 번 입력→출력과 달리 멈춤 조건을 만날 때까지 반복한다.

02 어떻게 동작

LLM이 환경을 관찰하고 다음 행동을 정해 도구를 쓰고, 그 결과를 다시 받아 판단을 잇는다. **사용자 지시**로 출발해 **독립적으로 계획·실행**하다 필요하면 사람에게 되묻는다.

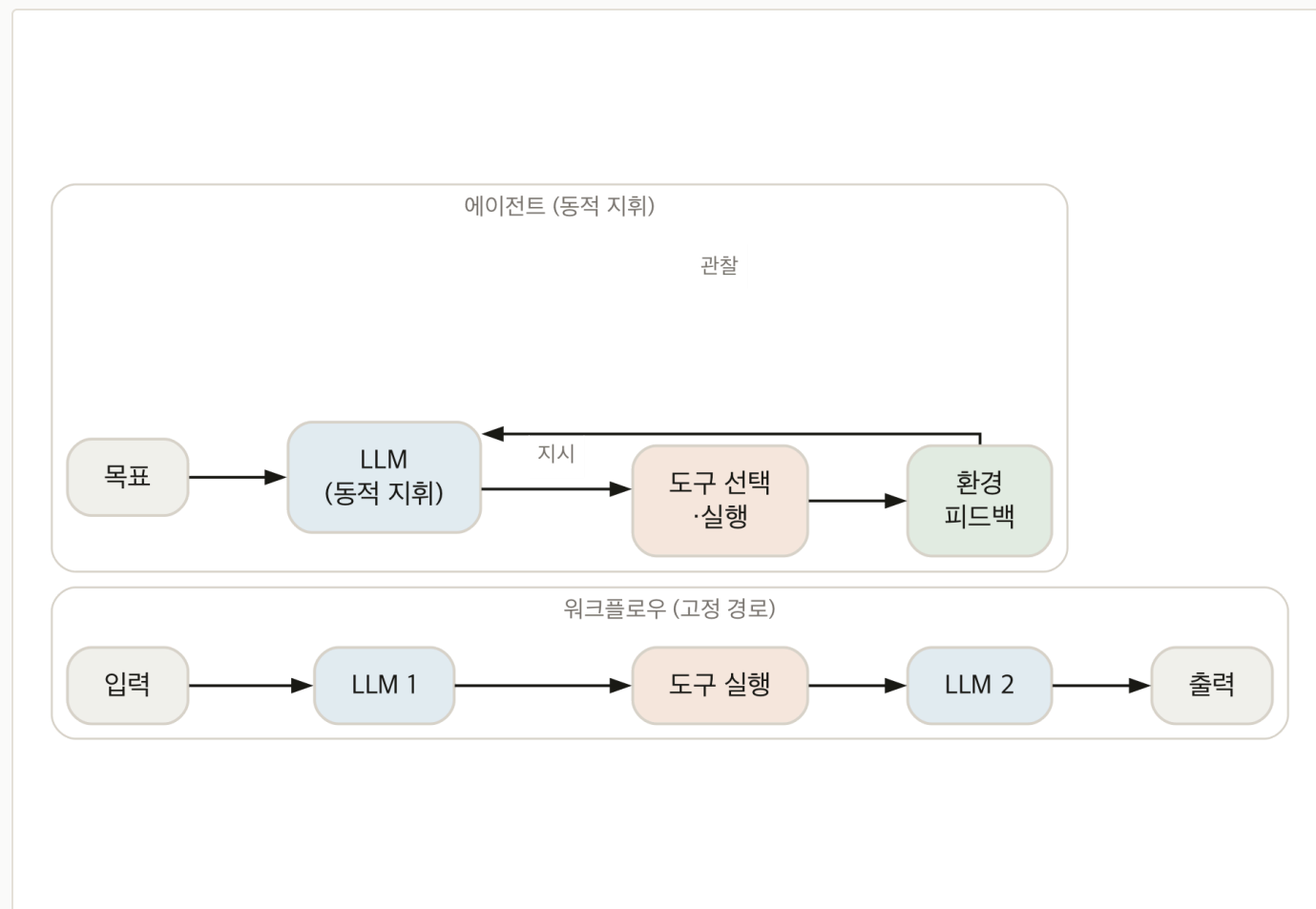
03 왜 중요

핵심 갈림길은 **단발 응답**이나, **목표 달성까지 루프**를 도는 **실행 주체**냐다. 에이전트는 어떻게 끝낼지를 **스스로 통제**한다.

04 어디에 쓰나

질문에 답하는 직원이 챗봇이라면, 일을 맡겨두면 끝내는 직원이 에이전트다.

워크플로우 vs 에이전트 — Anthropic의 구분



워크플로우와 에이전트의 구분. 직접 그린 도식

01 무엇을 보나

같은 LLM과 도구를 쓰더라도 **두 갈래**로 나뉜다. 위쪽은 정해진 경로를 따르는 **워크플로우**, 아래쪽은 스스로 경로를 정하는 **에이전트**다.

02 어떻게 동작

워크플로우는 LLM과 도구가 **미리 정해진 코드 경로**로 오케스트레이션된다. **에이전트**는 LLM이 자신의 프로세스와 도구 사용을 **동적으로 지휘**하며 어떻게 과업을 달성할지 통제한다.

03 왜 중요

둘은 우열이 아니라 **선택지**다. 잘 정의된 과업엔 워크플로우가 **예측가능성과 일관성**을, 유연성과 모델 주도 결정이 클 때는 에이전트가 더 낫다.

04 어디에 쓰나

경로를 미리 그릴 수 있으면 워크플로우로, 그릴 수 없으면 에이전트로 간다.

룰 기반 자동화에서 판단하는 자동화로

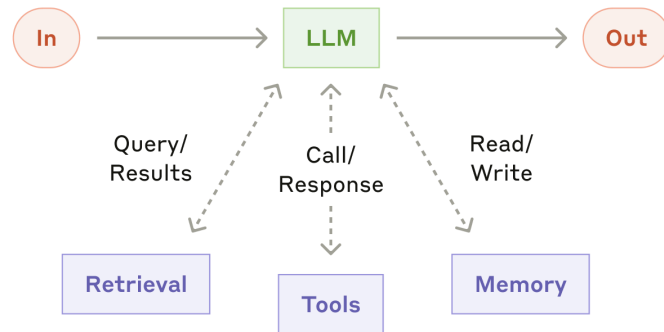
과거 자동화는 사람이 **모든 분기**를 미리 코딩해야 했다. 경우의 수가 늘면 규칙도 함께 늘어 한계가 분명했다.

달라진 것은 **두 가지 능력**이다. LLM은 텍스트를 이해·생성·추론하고, 동시에 **도구 호출(tool use)**로 외부 기능을 직접 부른다.

이 둘이 합쳐지면 사람이 모든 경우를 미리 정의하지 않아도 된다. 에이전트가 **상황을 판단해** 다음 행동을 고르기 때문이다.

그래서 반복 업무의 무인화, 여러 단계를 거치는 파이프라인 위임이 실제로 가능해졌다. **규칙을 짜는 자동화에서 판단하는 자동화**로 옮겨간 것이 핵심이다.

추론 + 도구 호출이 만나면



증강 LLM(검색·도구·메모리). 출처: Anthropic, Building Effective AI Agents, 2024

01 무엇을 보나

가운데 LLM이 **검색(Retrieval)·도구(Tools)·메모리(Memory)** 세 요소와 **양방향**으로 연결된 **증강 LLM** 구조다. 이것이 에이전트의 기본 단위다.

02 어떻게 동작

LLM은 검색으로 **질의-결과**를 주고받고, 도구로 **호출-응답**을 처리하며, 메모리에 **읽고 쓴다**. 추론만 하던 모델이 외부와 상호작용하는 능력을 얻는다.

03 왜 중요

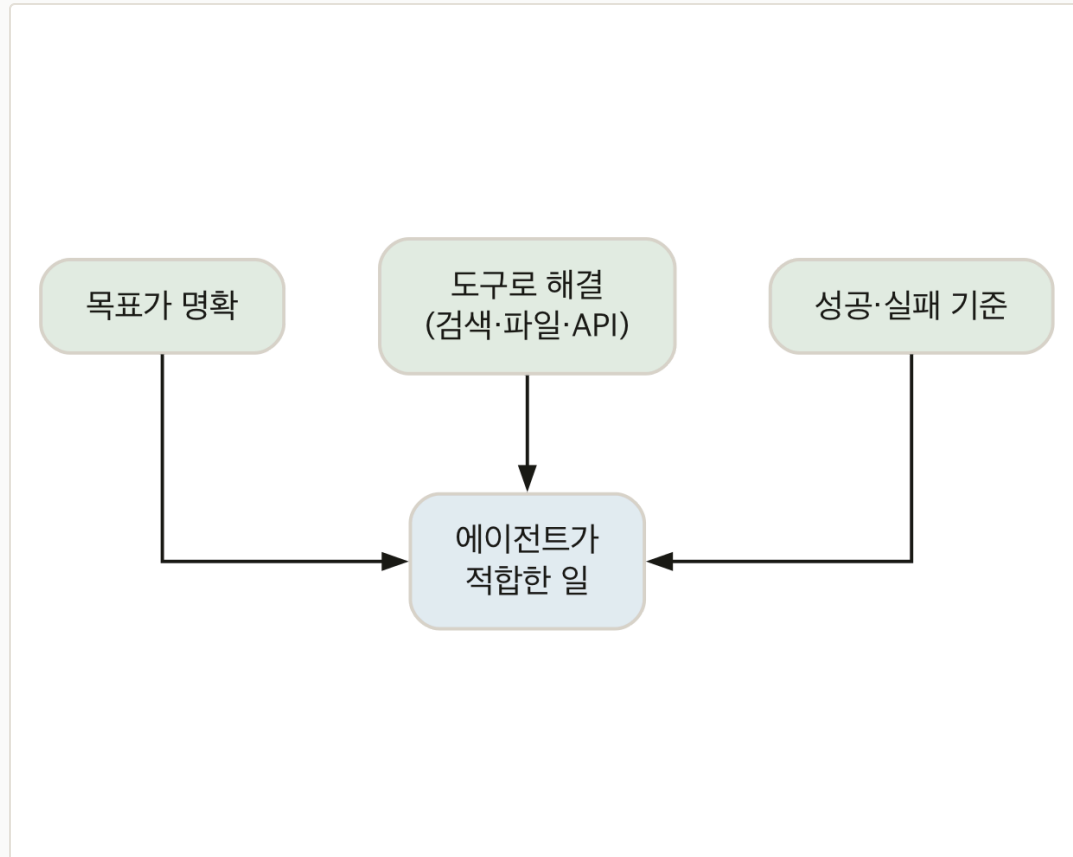
추론과 **도구 호출**이 한 자리에서 만나면, 모델은 생각만 하지 않고 **행동**할 수 있다. 관찰→판단→행동 루프가 성립하는 토대다.

04 어디에 쓰나

에이전트를 만들 때 먼저 이 한 단위를 잘 갖춘 뒤 단계를 늘려간다.

에이전트가 잘 맞는 일

에이전트가 적합한 작업은 **목표가 명확하고**, 도구로 해결 가능하며, **성공/실패 기준**이 있는 일이다.



01 반복 업무 무인화

매번 사람이 판단을 끼우기 **번거로운 작업**을 맡긴다. 같은 흐름이 반복되는 일일수록 효과가 크다.

02 사람 판단이 적은 파이프라인

수집→정제→검증→작성처럼 단계는 많지만 각 단계 판단이 **기계적인** 흐름에 잘 맞는다.

03 24시간 무인 운영

캘린더 이벤트나 cron으로 트리거하면 **사람 없이 완주**한다. 야간·주말에도 멈추지 않는다.

04 세 조건 점검

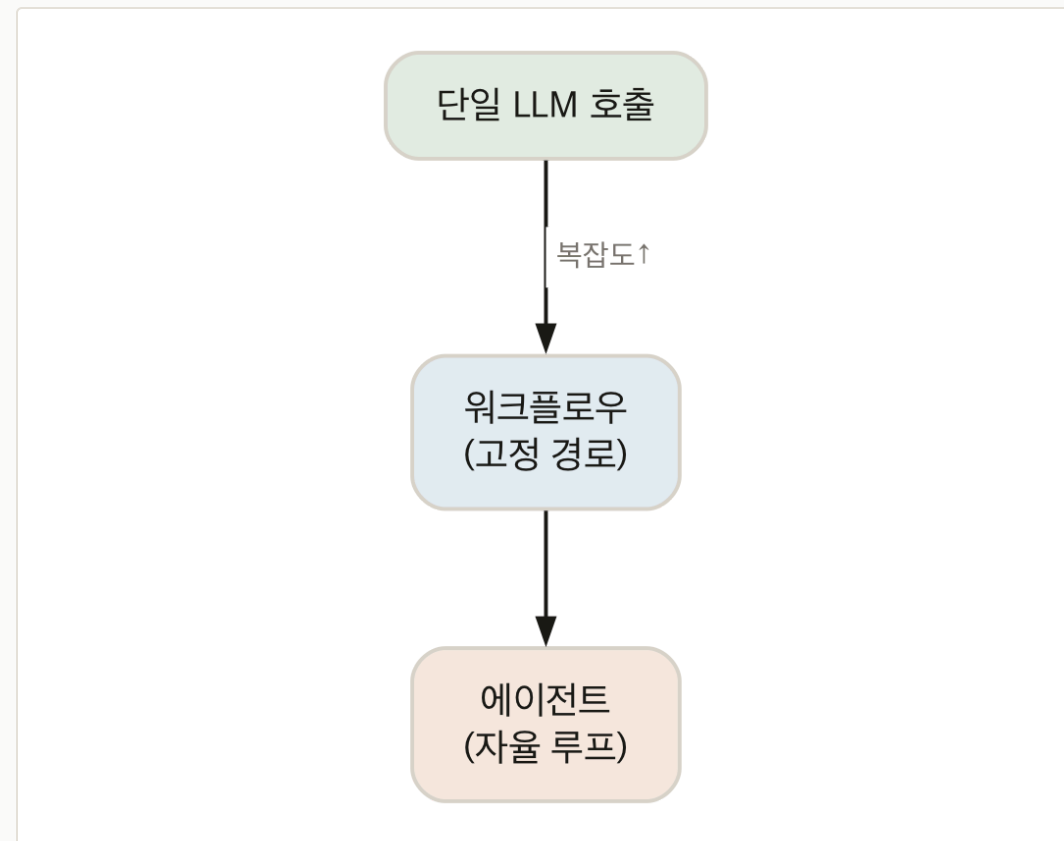
목표·도구·기준 셋이 모두 갖춰질 때 적합하다. 하나라도 모호하면 먼저 그것부터 정의한다.

에이전트가 항상 정답은 아니다

에이전트가 멋져 보여도 **단순한 작업**에는 비용·지연·환각만 늘린다. 도식의 사다리처럼 복잡도는 한 칸씩 올리는 것이 맞다.

Anthropic은 가능한 가장 단순한 해법을 먼저 찾고, 필요할 때만 복잡도를 올리라고 권한다. 많은 경우 검색과 예시를 곁들인 **단일 LLM 호출** 최적화만으로 충분하다.

판단 기준은 분명하다. 목표가 명확하고 단계가 정형이면 **단일 호출**이나 **워크플로우**가 맞고, 상황 판단과 다단계 자율성이 필요할 때 비로소 **에이전트**가 적합하다.



복잡도 사다리(단일 호출 → 워크플로우 → 에이전트). 직접 그린 도식

"가장 단순한 해법을 먼저, 복잡도는 필요할 때만."

에이전트를 이루는 다섯 조각

에이전트는 모델·도구·메모리·루프·환경 다섯 요소가 맞물려 동작한다.

에이전트 = 다섯 조각

모델 (두뇌·추론)

도구 (손·행동)

메모리 (상태 유지)

루프 (관찰→판단→행동)

환경 (외부 세계)

01 모델(두뇌)

LLM이 상황을 이해하고 다음 행동을 결정한다. 에이전트의 판단력이 여기서 나온다.

02 도구(손)

웹 검색, 파일 읽기/쓰기, API 호출, 코드 실행 등 모델이 직접 실행하는 능력이다.

03 메모리·상태

이전 단계 결과를 어떻게 기억해 다음에 넘기느냐다. 파일 핸드오프, 컨텍스트, 외부 저장소를 쓴다.

04 루프와 환경

관찰→판단→행동을 반복하는 흐름이 루프, 그 상대가 환경(파일·웹·API·DB)이다.

관찰 → 판단 → 행동, 그리고 다시 관찰

STEP 01

관찰 (Observation)

에이전트가 **환경의 현재 상태**를 읽는다. 직전 도구의 결과, 파일 내용, 사용자 입력이 모두 관찰 대상이다.

→

STEP 02

판단 (Reasoning)

모델이 관찰을 바탕으로 **무엇을 할지** 정한다. 도구를 부를지, 어떤 인자로 부를지, 아니면 끝낼지를 결정한다.

→

STEP 03

행동 (Action)

정한 대로 **도구를 실행**한다. 검색·파일 쓰기·API 호출 같은 실제 동작이 환경을 바꾼다.

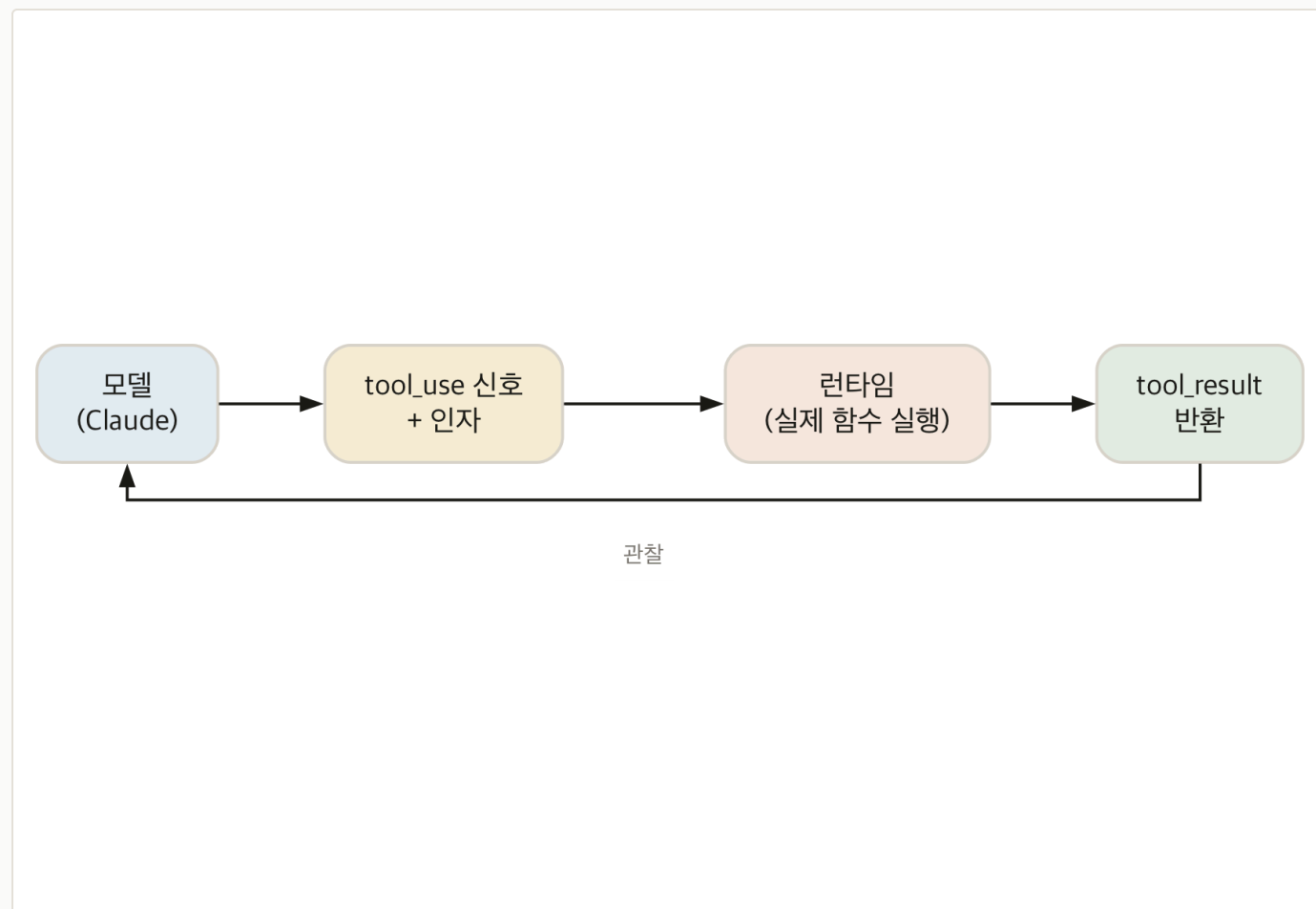
→

STEP 04

다시 관찰

행동의 결과를 또 관찰해 루프를 잇는다. **목표 달성**이나 **멈춤 조건**을 만날 때까지 반복한다.

도구 호출 루프는 실제로 이렇게 돈다



도구 사용 루프(tool_use → 실행 → tool_result). 직접 그린 도식

01 무엇을 보나

관찰→판단→행동을 **API 차원의 실제 신호**로 옮긴 도식이다.
모델과 내 코드(런타임) 사이를 도는 순환을 본다.

02 어떻게 동작

Claude가 응답을 `stop_reason: tool_use` 로 멈추고 **어떤 도구를 어떤 입력으로 부를지** 담은 **tool_use** 블록을 낸다. 내 코드가 그 함수를 실행한 뒤 결과를 **tool_result**로 돌려준다.

03 왜 중요

모델은 그 결과를 보고 다음 행동을 정한다. 이 **호출→실행→결과**
관찰→반복이 곧 에이전트 루프 그 자체다.

04 어디에 쓰나

`web_search` 같은 **서버 도구**는 Anthropic 쪽에서 실행돼
결과만 받고, 내 함수는 클라이언트 도구로 직접 처리한다.

설계는 다섯 단계로

STEP 01

목표 명확화

무엇을 끝낸 상태가 **성공**인지 한 문장으로 적는다. 모호하면 루프를 멈출 지점도 못 정한다.



STEP 02

입력·출력 정의

무엇을 받아 무엇을 내보낼지 정하면 다음 단계 쪼개기가 쉬워진다.



STEP 03

단계 분해

목표를 다룰 만한 작은 단위로 나눈다. 각 단계가 또렷해지고 디버그도 쉽다.



STEP 04

산출물 정의

단계마다 남길 **핸드오프 파일**을 정해, 어디서든 다시 돌릴 수 있게 한다.



STEP 05

게이트 위치

발행 직전처럼 **검증이 필요한 지점**에 통과 관문을 둔다.

단일 에이전트 vs 멀티에이전트

어느 쪽이 옳은 게 아니라, **과업의 성격**에 맞춰 고른다. 작게 시작해 필요할 때 나눈다.

단일 에이전트

하나가 전 단계를 처리

구조

하나의 에이전트가 수집부터 마무리까지 전 과정을 맡는다. 구성이 단순하다.

장점

빠르게 시작할 수 있고 흐름을 따라가기 쉽다. 단계가 적은 작업에 잘 맞는다.

한계

일이 커지면 **역할 경계가 흐려지고** 한 곳의 실패가 전체에 번지기 쉽다.

적합

단계가 적고 판단이 단순한 일. 입문 단계의 첫 에이전트로 권한다.

멀티에이전트

오케스트레이터가 분배

구조

메인 오케스트레이터가 여러 **서브에이전트**를 호출·분배한다. 메인은 진행만 맡는다.

장점

역할이 나뉘어 **병렬 처리**와 **재시도**가 쉽다. 단계별 책임이 뚜렷하다.

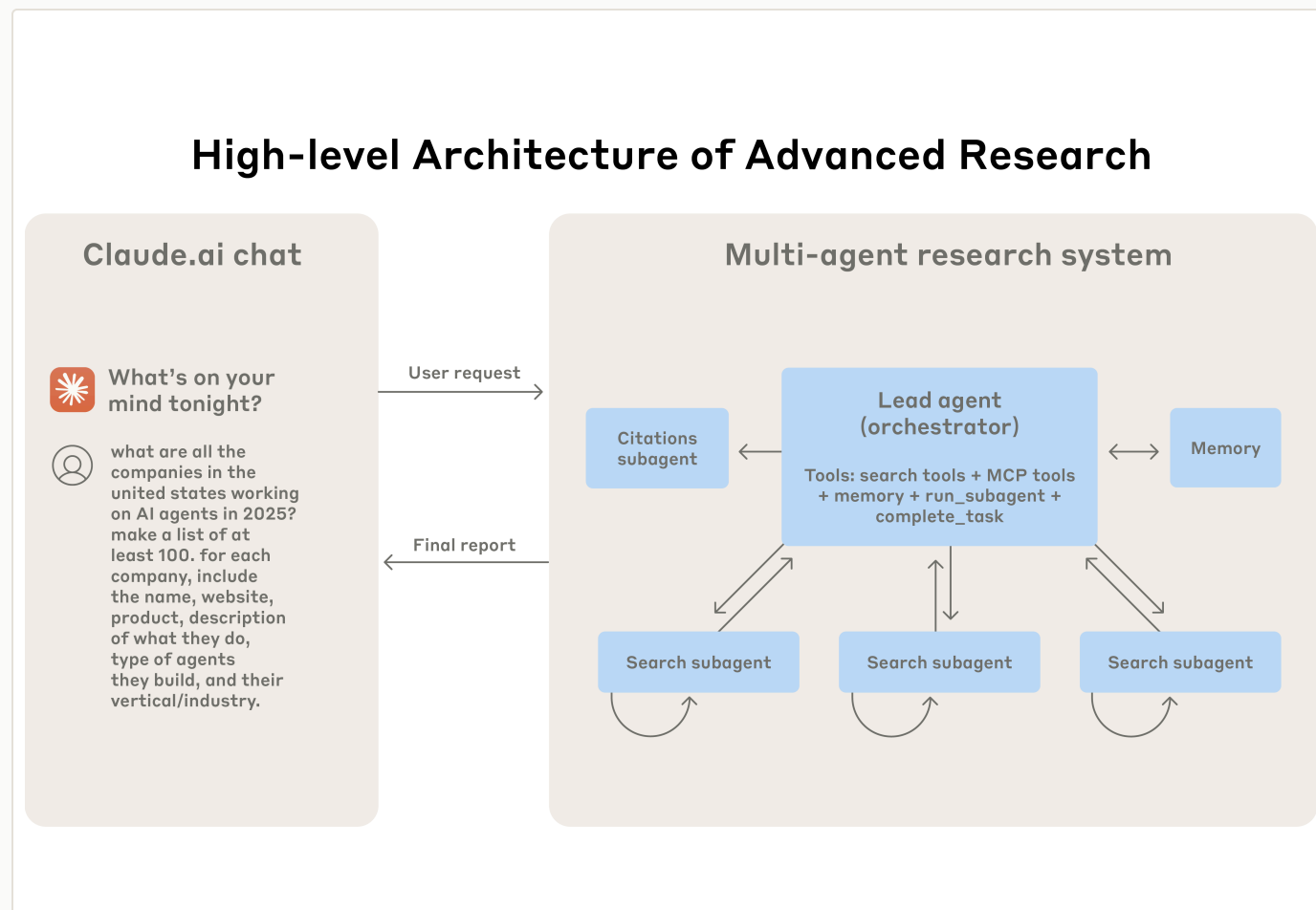
한계

구성이 복잡해 처음부터 크게 짜면 **디버그가 어렵다**. 토큰 소비도 늘어난다.

적합

단계가 많고 역할 분리·병렬화가 **값어치를 할 때**. 병목이 보이면 그때 나눈다.

리드가 조율하고 워커가 병렬로 일한다



01 무엇을 보나

오케스트레이터-워커 패턴의 구조도다. 가운데 리드 에이전트가 있고, 그 아래 전문 **서브에이전트(워커)**들이 붙는다.

02 어떻게 동작

리드가 **전체 과정을 조율**하며 작업을 쪼개 워커들에게 **병렬로 위임**한다. 워커들이 동시에 일한 결과를 리드가 종합한다.

03 왜 중요

하위 작업을 미리 다 예측할 수 없을 때, 리드가 **상황에 따라 동적으로** 나눠 맡길 수 있다. 단일 에이전트보다 넓은 일을 다룬다.

04 어디에 쓰나

Anthropic의 Research 시스템이 이 구조다. 리드는 진행을, 워커는 실제 조사를 맡는 식으로 책임을 나눈다.

오케스트레이터-워커 멀티에이전트 구조. 출처: Anthropic, How we built our multi-agent research system, 2025

멀티에이전트는 얼마나 강한가

단일 한 대로 벅찬 일을 역할 분담으로 푼다.

90.2%

멀티 > 단일

리드+서브에이전트 구성이 단일 모델보다 정확했다
(Anthropic 내부 평가, 2025). 일을 나눠 맡길수록 한
모델의 한계를 덜 탄다.

1리드

오케스트레이터

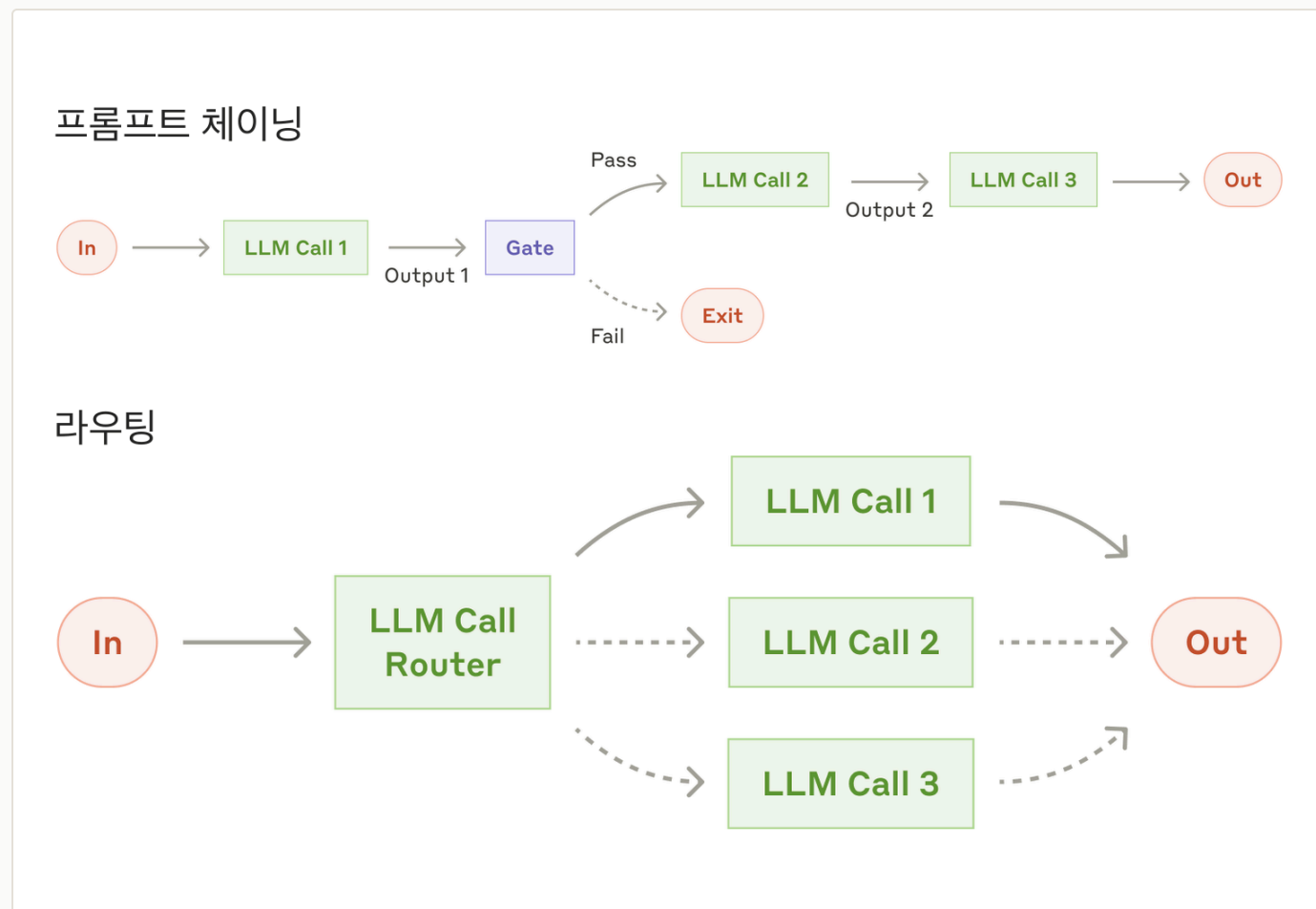
한 리드가 전체를 조율하고 **서브에이전트**에 위임한다. 누가
무엇을 맡을지 정하는 역할이 핵심이다.

병렬

동시 처리

서로 독립인 작업은 **동시에** 굴러 시간을 줄인다. 의존이 있으면
차례로 둔다.

프롬프트 체이닝 vs 라우팅



출처: Anthropic, Building Effective AI Agents (2024) — 위 프롬프트 체이닝·아래 라우팅

01 프롬프트 체이닝

입력이 여러 LLM 호출을 차례로 거쳐 출력에 닿는다. 중간 게이트로 통과·탈락을 가른다.

02 라우팅

라우터가 입력을 먼저 분류해 전문 경로 중 하나로 보낸다. 범주가 뚜렷할 때 정확도가 오른다.

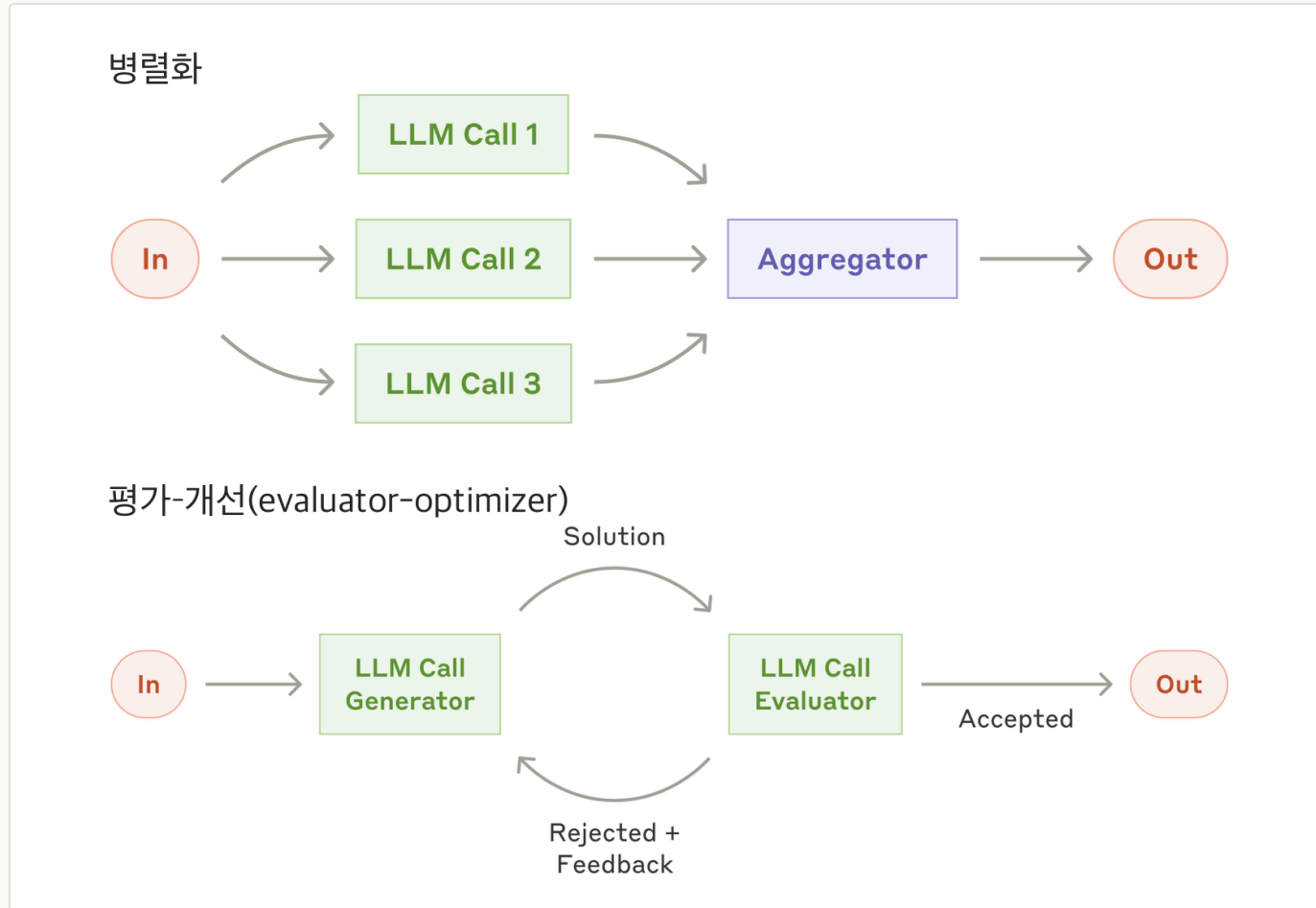
03 무엇이 다른가

체이닝은 한 줄로 차례차례, 라우팅은 갈래로 나눠 보낸다. 둘 다 흐름을 사람이 고정하는 워크플로우다.

04 어디에 쓰나

초안→검토→다듬기는 체이닝, 문의 유형별 분기는 라우팅으로.

병렬화 vs 평가-개선(evaluator-optimizer)



출처: Anthropic, Building Effective AI Agents (2024) — 위 병렬화·아래 평가-개선

01 병렬화

한 입력을 여러 LLM이 동시에 처리하고 결과를 **취합**한다.
나누거나 여러 번 돌려 속도·신뢰를 얻는다.

02 평가-개선(evaluator-optimizer)

생성 LLM과 평가 LLM이 피드백을 주고받으며 **반복 개선**한다.
기준이 또렷할 때 잘 수렴한다.

03 무엇이 다른가

병렬화는 **동시에 펼쳐** 모으고, 평가-개선은 **만들고 고치기**를 반복한다.

04 어디에 쓰나

긴 문서 분할·다수결은 병렬화, 번역·글 다듬기는 평가-개선으로.

결정론과 LLM을 나누어라

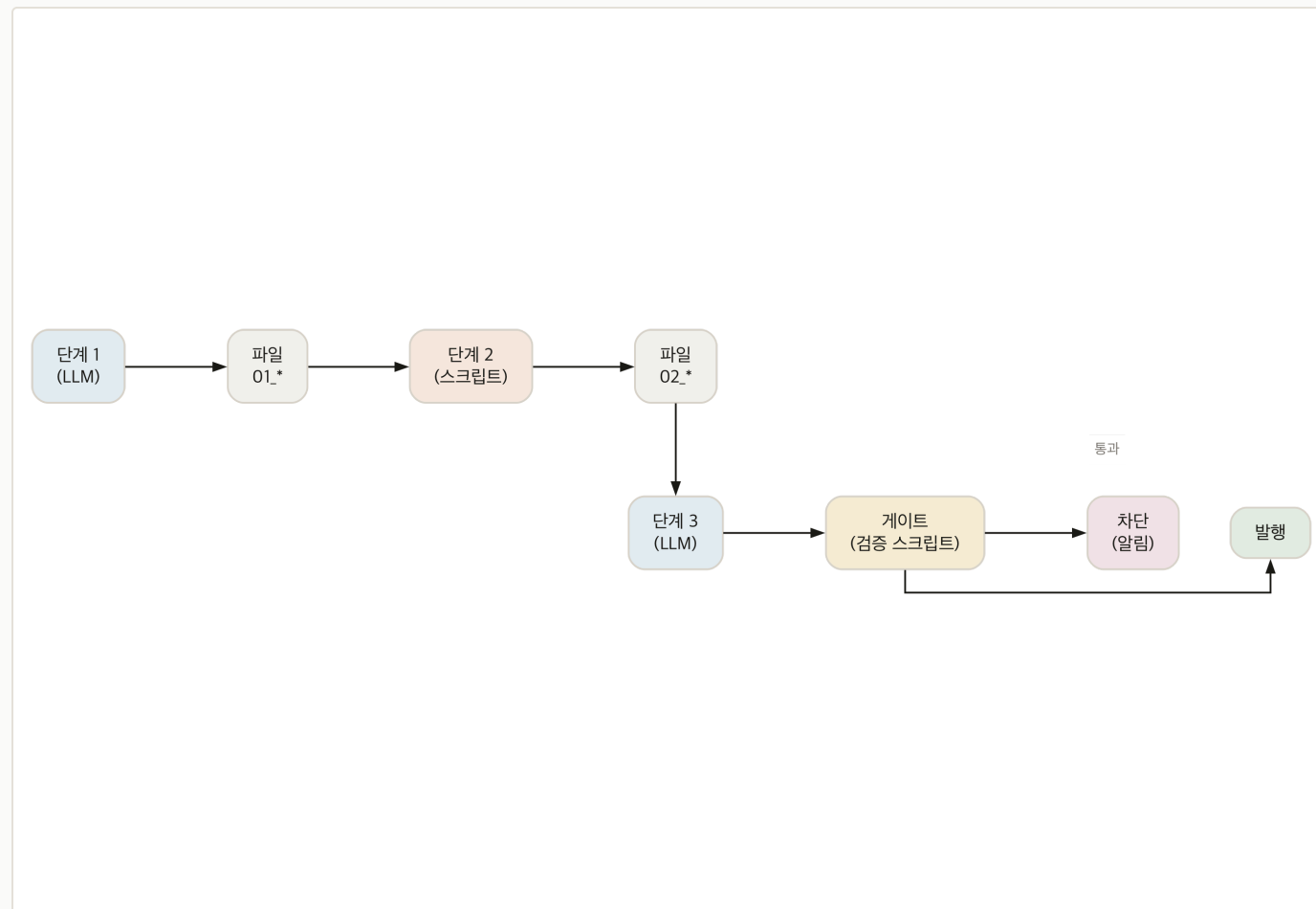
잘 도는 에이전트의 공통점은 **역할 분담**이다. 사람 판단이 필요 없는 일은 코드로, 생성과 판단만 LLM에 맡긴다.

필터·중복제거·형식 변환·게이트처럼 답이 정해진 일은 **스크립트(결정론)**로 처리한다. 빠르고 싸고 **틀리지 않는다**.

사실 판단·글쓰기·선별처럼 사람의 감각이 필요한 일만 **LLM**에 맡긴다. 결정론으로 될 일까지 LLM에 시키면 환각·비용·속도 문제가 생긴다.

또 하나는 **파일 핸드오프**다. 단계 간 데이터를 변수로 넘기지 않고 파일로 남기면 **어느 단계든 다시 실행**할 수 있다.

파일 핸드오프와 발행 전 게이트



직접 그린 도식 — 단계마다 검사하고, 발행 전 게이트에서 통과/반려를 가른다.

01 파일 핸드오프

각 단계가 `01_`, `02_` 같은 파일을 남긴다. 메모리 공유 없이 데이터가 흐른다.

02 재실행 가능

어느 단계든 그 파일부터 다시 시작할 수 있다. 중간에 멈춰도 처음으로 돌아가지 않는다.

03 발행 전 게이트

마지막에 검증 스크립트가 구조와 사실을 점검한다. 통과해야 다음으로 간다.

04 차단의 의미

게이트는 환각·빈 슬롯을 발행 전에 막는다. 통과 못 하면 멈추고 사유를 남긴다.

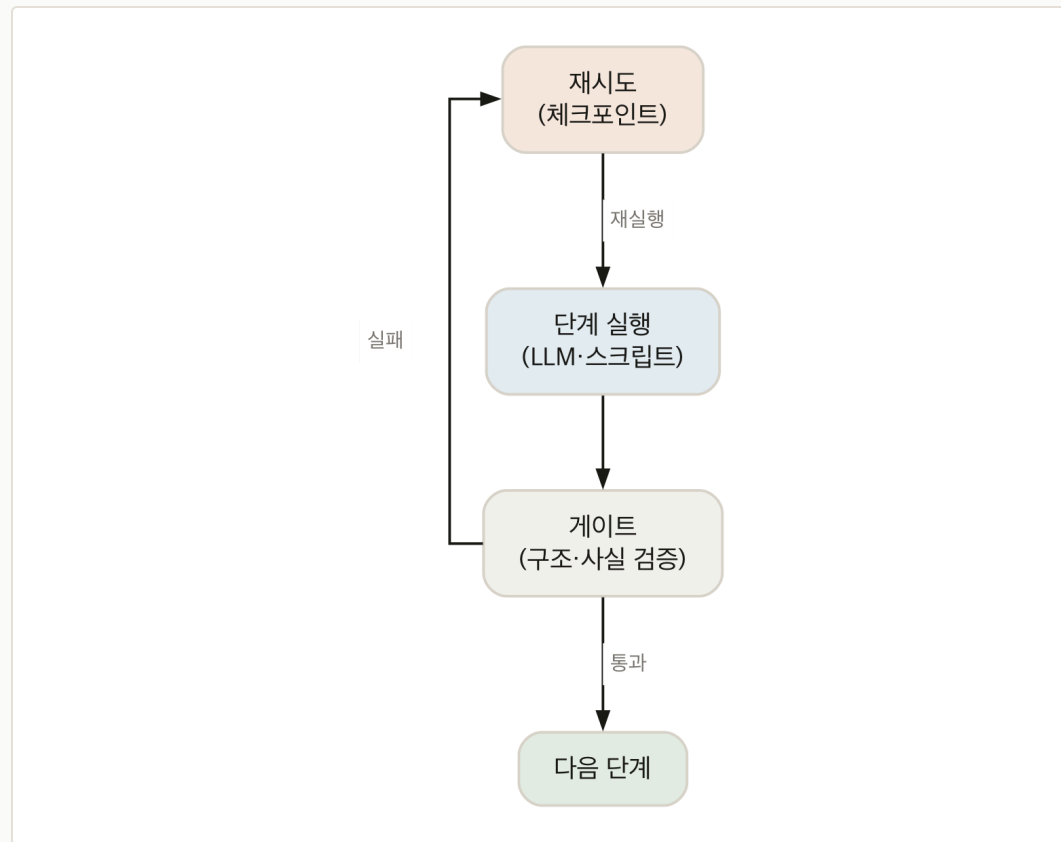
오래 도는 에이전트엔 결정론적 안전장치를

에이전트는 여러 도구 호출에 걸쳐 오래 실행된다.

그래서 작은 실패가 쌓이면 치명적이 될 수 있다.

Anthropic은 이를 막기 위해 AI 에이전트의 유연성에 재시도 로직과 정기 체크포인트 같은 결정론적 안전장치를 결합한다고 밝힌다.

도식의 흐름처럼 단계마다 점검을 두면, 실패한 지점에서 그 체크포인트부터 재시도할 수 있다. 게이트와 재실행 가능한 핸드오프가 곧 운영 신뢰성의 정석인 이유다.

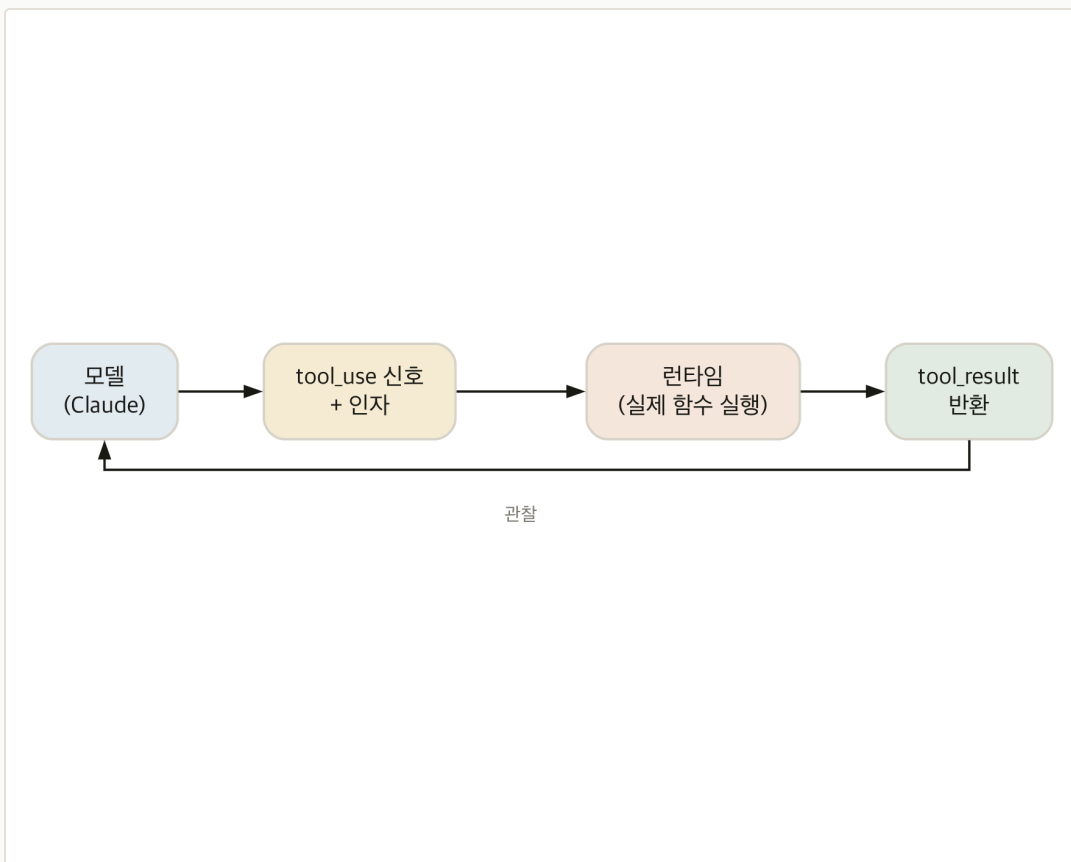


단계 → 게이트 → 통과/실패 재시도. 직접 그린 도식

"작은 실패가 쌓이기 전에, 체크포인트에서 되돌린다."

tool use는 모델의 손이 된다

도구 사용(tool use)은 모델이 외부 기능을 직접 부르게 하는 메커니즘이다. 도구가 없으면 에이전트는 **관찰만 하고 행동하지 못한다**.



01 어떻게 부르나

모델이 **이 도구를 이 인자로 호출하라**는 신호를 내면 런타임이 **실제 함수를 실행**하고 결과를 돌려준다.

02 도구 정의

이름(name)·설명(description)·**input_schema**(JSON Schema)로 선언한다. 모델은 이것 보고 언제 어떻게 쓸지 판단한다.

03 설명이 핵심

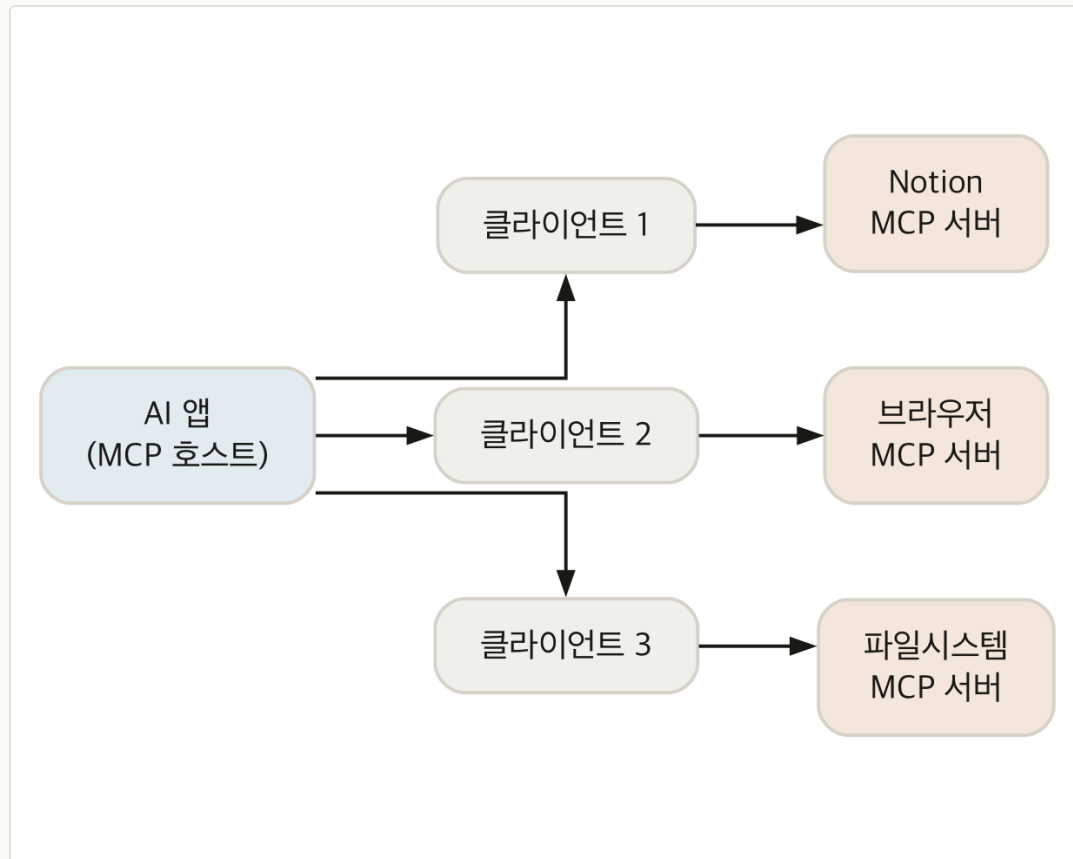
상세한 설명이 도구 성능을 좌우한다. 무엇을·언제·어떤 파라미터로 쓰는지 도구마다 **3~4문장 이상** 적기를 권한다.

04 스스로 결정

도구를 주면 기본값 `auto` 에서 모델이 **호출 여부를 직접** 정한다. 답이 컨텍스트에 없을 때 도구를 부른다.

MCP — 도구를 표준으로 낚는다

MCP(Model Context Protocol)는 데이터 소스와 AI 도구를 잇는 **오픈 표준**이다. 도구를 플러그인처럼 붙인다.



01 푸는 문제

도구를 하나씩 직접 이으면 **모델 M개 × 소스 N개**마다 커넥터가 필요한 **N×M 통합** 문제가 생긴다.

02 하나의 표준

MCP는 이 연결을 **하나의 표준**으로 통일한다. 새 도구를 표준 방식 하나로 낚으면 된다.

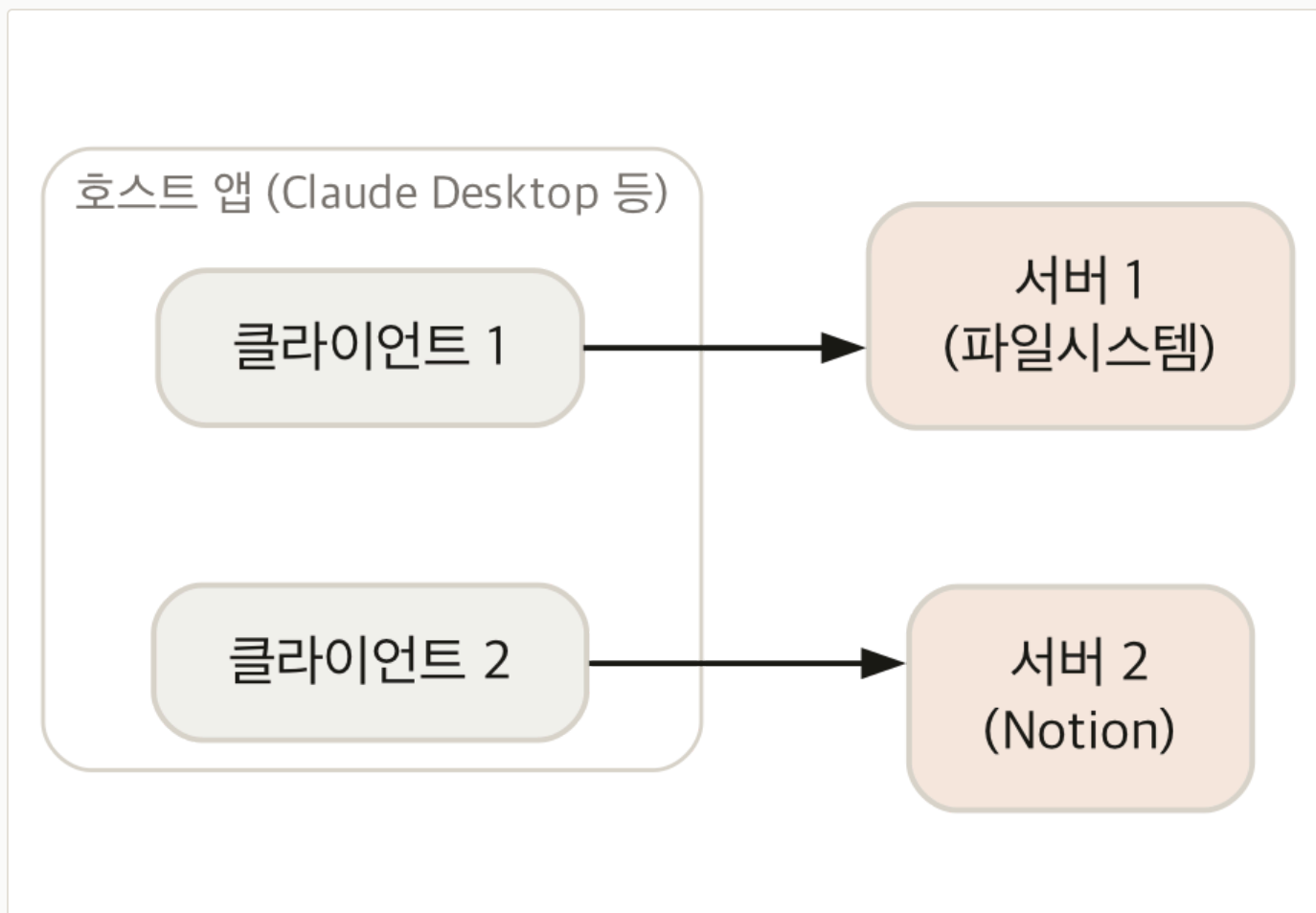
03 연결 구조

AI 앱(호스트)이 서버마다 **클라이언트를 하나씩** 만들어 **1:1 전용 연결**을 유지한다. Notion·브라우저·파일시스템 서버를 같은 방식으로 붙인다.

04 확산

2024년 11월 25일 Anthropic이 오픈소스로 공개했고, 2025년 3월 **OpenAI**가 채택하며 번졌다.

호스트 · 클라이언트 · 서버



MCP 호스트-클라이언트-서버 구조. 직접 그린 도식

01 무엇을 보나

MCP의 세 역할이다. AI 앱인 **호스트** 안에 여러 **클라이언트**가 있고, 각 클라이언트가 바깥 **서버**에 1:1로 연결된다.

02 어떻게 동작

호스트는 여러 클라이언트를 조율·관리하고, **클라이언트**는 서버와 전용 연결을 유지하며 맥락을 받아오고, **서버**는 클라이언트에 맥락을 제공한다.

03 왜 중요

서버가 노출하는 구성요소는 **세 가지**다. **Tools**(실행 함수), **Resources**(맥락 데이터), **Prompts**(재사용 템플릿).

04 어디에 쓰나

Notion MCP를 붙였다는 곧 호스트가 그 서버용 클라이언트를 띄워 tools·resources·prompts를 받는다는 뜻이다.

스킬은 필요할 때만 로드된다

STEP 01

1단계 · 이름과 설명만

평소엔 **name·description**만 올려 둔다. 에이전트는 **언제 이 스킬을 쓸지**만 알고 컨텍스트는 거의 안 쓴다.



STEP 02

2단계 · SKILL.md 본문

작업과 맞으면 그때 **SKILL.md** 본문을 통째로 읽어 절차·예시를 펼친다.



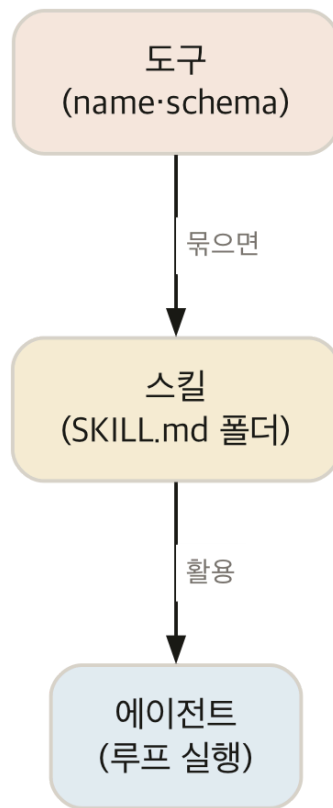
STEP 03

3단계 · 번들 파일

더 필요하면 **번들 파일**까지 탐색한다. 이 **점진적 공개**로 스킬이 많아도 **확장** 가능하다.

도구 · 스킬 · 에이전트 한 줄 정리

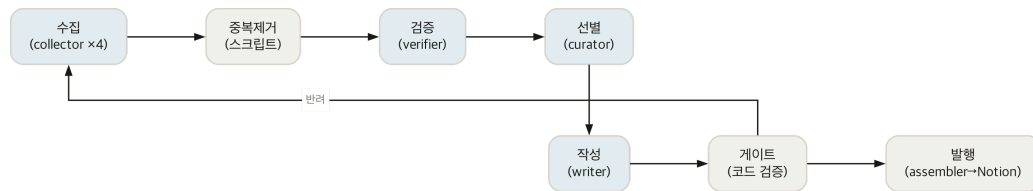
셋은 **포함 관계**다 — 도구(기능)를 스킬(방법론 묶음)이 묶고, 에이전트(행위자)가 그것을 활용한다.



- 01 도구 = 기능**
모델이 호출하는 **단일 기능**이다. 이름·설명·input_schema(JSON Schema)로 선언한다.
- 02 스킬 = 방법론 묶음**
SKILL.md 폴더에 지침·스크립트·리소스를 묶은 능력 모듈이다. 에이전트가 **동적으로 발견·로드**한다.
- 03 에이전트 = 행위자**
도구와 스킬을 **루프 안에서 활용**해 목표를 달성하는 **실행 주체**다.
- 04 한 줄 정리**
도구는 기능, 스킬은 방법론, 에이전트는 행위자. 헛갈릴 때 이 한 줄로 돌아온다.

ai-news-agent — 무인 뉴스 다이제스트

ai-news-agent는 최근 14일 AI 뉴스를 수집→검증→선별→작성→발행까지 사람 개입 없이 완주하는 멀티에이전트 시스템이다.



01 무엇을 하나

캘린더 **ai뉴스** 이벤트로 트리거돼 뉴스를 모아 **Notion**에 자동 발행한다. D-3~D-1엔 누적 수집, D 당일엔 발행.

02 메인은 분배만

오케스트레이터는 직접 일하지 않고 **5개 서브에이전트** (collector·verifier·curator·writer·assembler)를 **호출·분배·진행**만 한다.

03 결정론은 코드로

중복제거·윈도우 필터·게이트는 **Python** 스크립트가, 수집·검증·선별·작성은 LLM이 맡는다.

04 파일로 핸드오프

단계 간 데이터는 `output/<호>/NN_*` 파일로 넘긴다. **어느 단계든 재실행** 가능하다.

수집에서 발행까지 7단계

STEP 01

1. collector (병렬 4)

소스군 A~D를 동시에 수집해

`01_raw`로 남긴다. 속도가 중요해

`haiku` 모델을 쓴다.



STEP 02

2. 결정론 필터

`dedup.py` · `window_filter.py`가

중복을 제거하고 14일 윈도우로 거른다.

코드로 처리한다.



STEP 03

3. verifier → 4. curator

교차검증으로 수치 사실을 확인

(`03_verified`)한 뒤 심층

3개를 선별한다. 정확도를

위해 `opus`.



STEP 04

5. writer → 6. 게이트

본문을 쓰되 수치는

`03_verified`에서만 인용한다.

`check_draft.verify_claims`

게이트를 통과해야 한다.



STEP 05

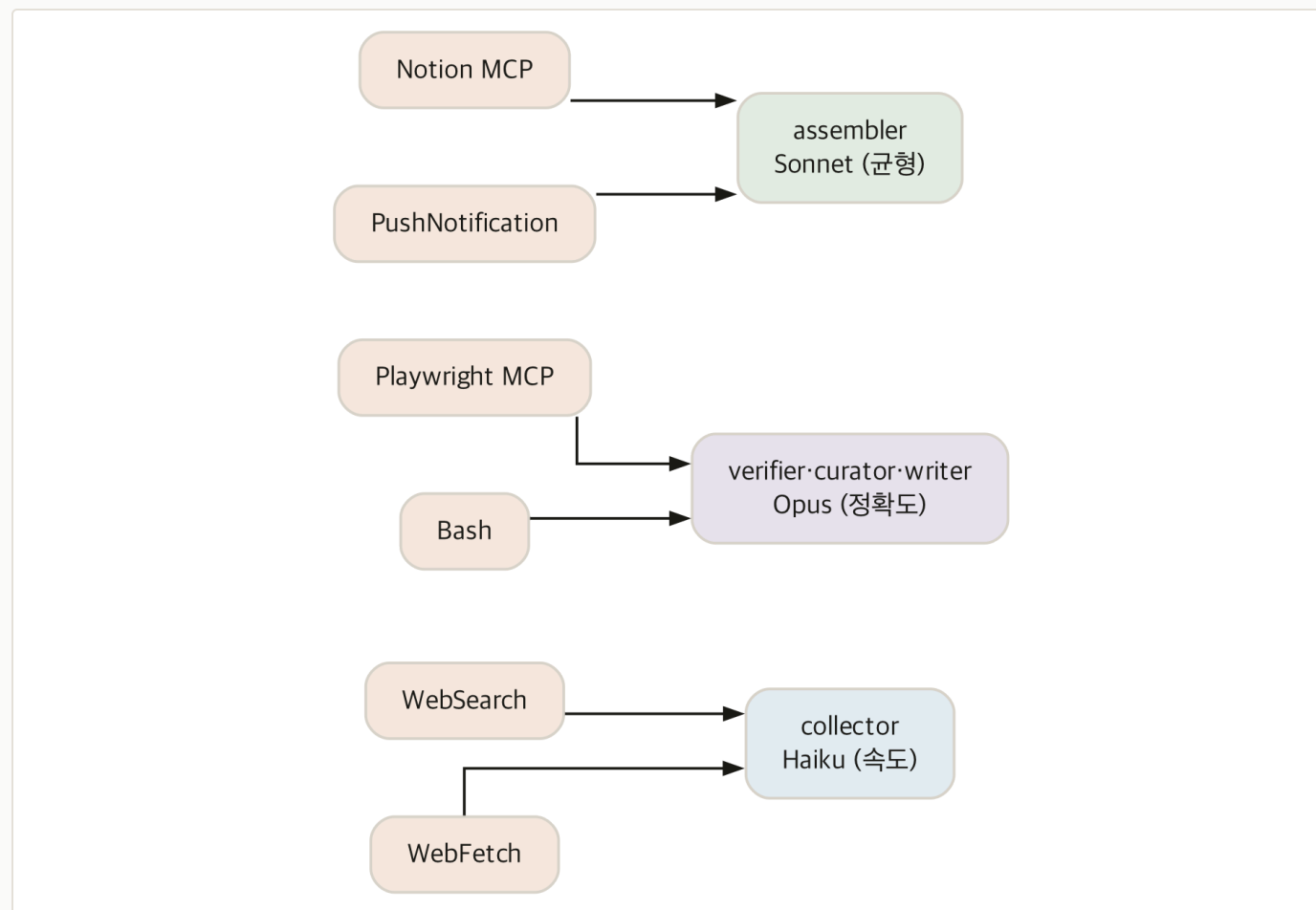
7. assembler → 발행

Notion에 발행하고 아카이브에

누적한 뒤 완료 알림을 보낸다.

조립은 `sonnet`.

어떤 도구를, 어느 모델에



도구 배정과 모델 차등. 직접 그린 도식

01 무엇을 보나

도구 배정과 모델 차등을 한 그림에 담았다. 어떤 도구를 쓰고, 어느 에이전트에 어느 모델을 두는지 본다.

02 어떻게 동작

도구는 **WebSearch·WebFetch·Bash**에 **Playwright MCP**(동적·유료 페이지 우회), **Notion MCP**(발행), **PushNotification**(알림)을 더해 쓴다.

03 왜 중요

에이전트마다 모델을 다르게 둔다. **collector=haiku**(속도), **verifier·curator·writer=opus**(정확도), **assembler=sonnet**.

04 어디에 쓰나

일의 성격에 모델을 맞추면 **속도와 정확도, 비용**의 균형을 잡을 수 있다.

흔한 함정 vs 권장 습관

대부분의 사고는 **익숙한 함정**에서 나온다. 반대편의 습관으로 바꾸면 대부분 막힌다.

흔한 함정

이렇게 하면 무너진다

모든 걸 LLM에

필터·정렬·형식 변환까지 LLM에 시키면 **환각·비용·속도** 문제가 터진다.

메모리 공유

에이전트끼리 **변수로 데이터**를 넘기면 재실행이 안 되고 디버그가 어렵다.

게이트 없이 발행

검증 없이 곧장 내보내면 **환각·빈 슬롯**이 그대로 노출된다.

처음부터 크게

다짜고짜 **복잡한 멀티에이전트**를 짜면 어디가 틀렸는지 찾기 어렵다.

권장 습관

이렇게 하면 버틴다

결정론은 코드로

답이 정해진 일은 **스크립트**로, **생성·판단**만 LLM에 맡긴다.

파일로 핸드오프

단계 간 데이터를 **파일**로 남겨 **언제든 재실행** 가능하게 둔다.

차단형 게이트

발행 전 **검증 스크립트**로 구조·사실을 점검해 통과해야 다음으로 간다.

작게 시작

단일 에이전트로 출발해 **병목이 보이면** 그때 서브에이전트로 나눈다.

핵심 3가지와 다음 단계

잘 만드는 핵심 3가지

- ▶ **파일 핸드오프** — 단계 간 데이터를 파일로 남겨 **어느 단계든 재실행** 가능하게.
-

- ▶ **결정론과 LLM 분리** — 정해진 일은 코드로, 생성·판단만 LLM에.
-

- ▶ **발행 전 게이트** — 검증을 통과해야 다음으로. **환각·빈 슬롯**을 미리 막는다.

다음 단계로

- ▶ **단일 에이전트** 직접 구현 — 도구 하나 붙여보기.
-

- ▶ **멀티에이전트**로 확장 — 오케스트레이터 + 서브에이전트.
-

- ▶ **MCP 도구** 연결 실습, 그리고 **실제 시스템**(ai-news-agent 같은) 분석.

에이전트는 **목표를 받아 도구를 쓰며 루프를 도는** 자율 실행 주체다. 작게 시작해 한 단계씩 키운다.